

Practical - 1

Aim: To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

1) Bubble Sort

Algorithm:

Input : An unsorted array $A[0 \dots n-1]$
Output : Sorted array A in ascending order

Algorithm

Step 1 :- Read array A and number of elements n .
Step 2 :- Compare $A[j]$ and $A[j+1]$
Step 3 :- If $A[j] > A[j+1]$,
 Swap $A[j]$ and $A[j+1]$
Step 4 :- Repeat until end of array
Step 5 :- Repeat for remaining elements
Step 6 :- Continue until array is sorted
Step 7 :- Print sorted array
Step 8 :- End

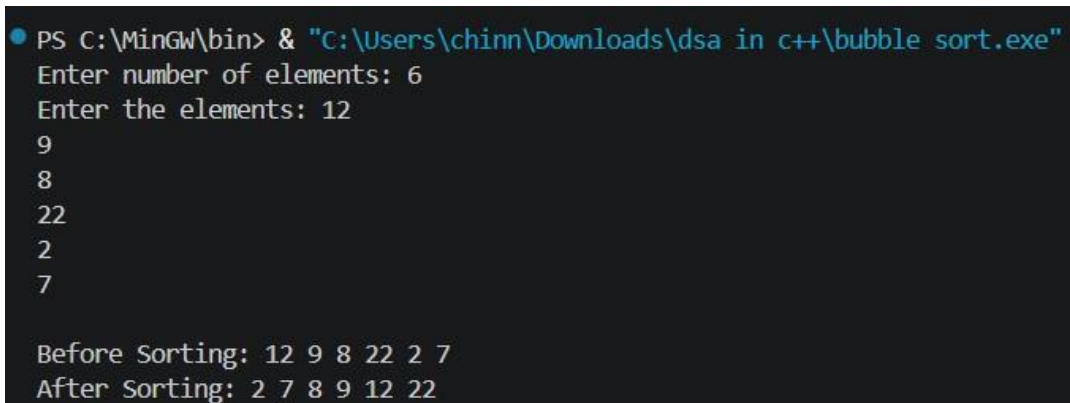
Program:

```
#include <iostream>
using namespace std;
class BubbleSort
{
public:
    int arr[100], n;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
    void printArray(){
        for (int i = 0; i < n; i++){
            cout << arr[i] << " ";
        }
    }
};
```

```
cout << endl;
}
void logic(){
    int temp;
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
};
int main()
{
    BubbleSort bs;
    bs.arrayInput();
    cout << "\nBefore Sorting: ";
    bs.printArray();
    bs.logic();
    cout << "After Sorting: ";
    bs.printArray();
    return 0;
}
```

Output:



```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\bubble sort.exe"
Enter number of elements: 6
Enter the elements: 12
9
8
22
2
7

Before Sorting: 12 9 8 22 2 7
After Sorting: 2 7 8 9 12 22
```

Time Complexity:

Time complexity

for ($i=0; i < n-1; i++$) { $\rightarrow O(n)$
 for ($j=0; j < n-1; j++$) { $\rightarrow O(n)$
 if ($A[j] > A[j+1]$) { $\rightarrow O(1)$
 swap($A[j], A[j+1]$); $\rightarrow O(1)$
 }
}

$T(n) = n \times n = n^2 = O(n^2)$
 Best case = $O(n^2) \rightarrow$ no swap required
 worst case $\rightarrow$ many swaps are required

$(n-1) + (n-2) + (n-3) + \dots + 1$
 $T(n) = 1 + 2 + 3 + \dots + (n-1)$
 $= \frac{n(n-1)}{2}$
 $T(n) = \frac{n^2 - n}{2}$
 $T(n) = O(n^2)$

Average case $\rightarrow$ some element need swapping and some don't, but both loops still run completely
 $T(n) = O(n^2)$
 Average case = $O(n^2)$

2) Selection sort:

Algorithm:

Input : An unsorted array $A[0 \dots n-1]$
 Output : Sorted array A in ascending order

Step 1 :- Read array A and number of elements n
 Step 2 :- set $min = i$
 Step 3 :- compare $A[min]$ with $A[j]$
 Step 4 :- if $A[j] < A[min]$,
 set $min = j$
 Step 5 :- Repeat steps 3 and step 4
 until the end of array
 Step 6 :- swap $A[i]$ and $A[min]$
 Step 7 :- Repeat the above steps
 for remaining elements
 Step 8 :- print sorted array
 Step 9 :- end

Program:

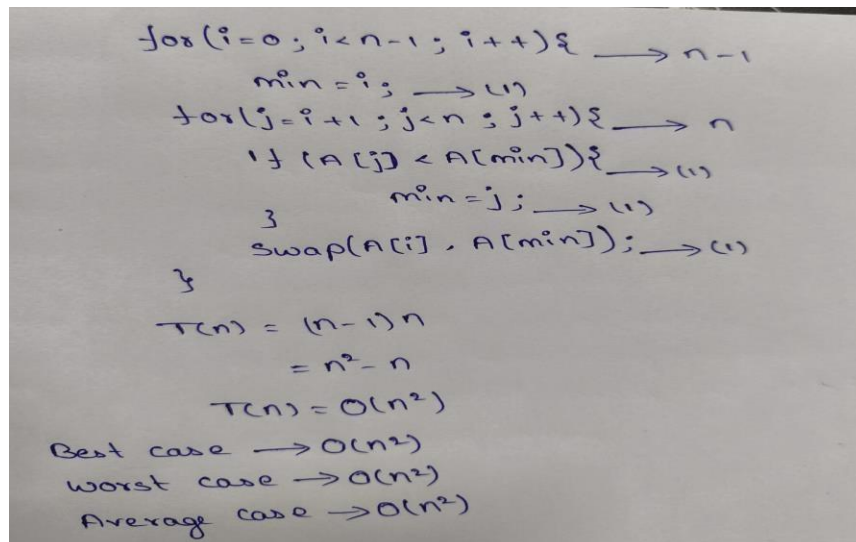
```
#include <iostream>
using namespace std;
class SelectionSort{
public:
    int arr[100], n;
    void arrayInput(){
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++){
            cin >> arr[i];
        }
    }
    void logic(){
        int i, j, min, temp;
        for (i = 0; i < n - 1; i++){
            min = i;
            for (j = i + 1; j < n; j++){
                if (arr[j] < arr[min]){
                    min = j;
                }
            }
        }
        Swap(arr[i], arr[min]);
    }
    void printArray() {
        for (int i = 0; i < n; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
}
```

```
};  
int main()  
{  
    SelectionSort ss;  
    ss.arrayInput();  
    cout << "Before Sorting: ";  
    ss.printArray();  
    ss.logic();  
    cout << "After Sorting: ";  
    ss.printArray();  
    return 0;  
}
```

Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\selectionsort.exe"  
Enter number of elements: 6  
Enter the elements: 12  
9  
2  
8  
31  
1  
Before Sorting: 12 9 2 8 31 1  
After Sorting: 1 2 8 9 12 31
```

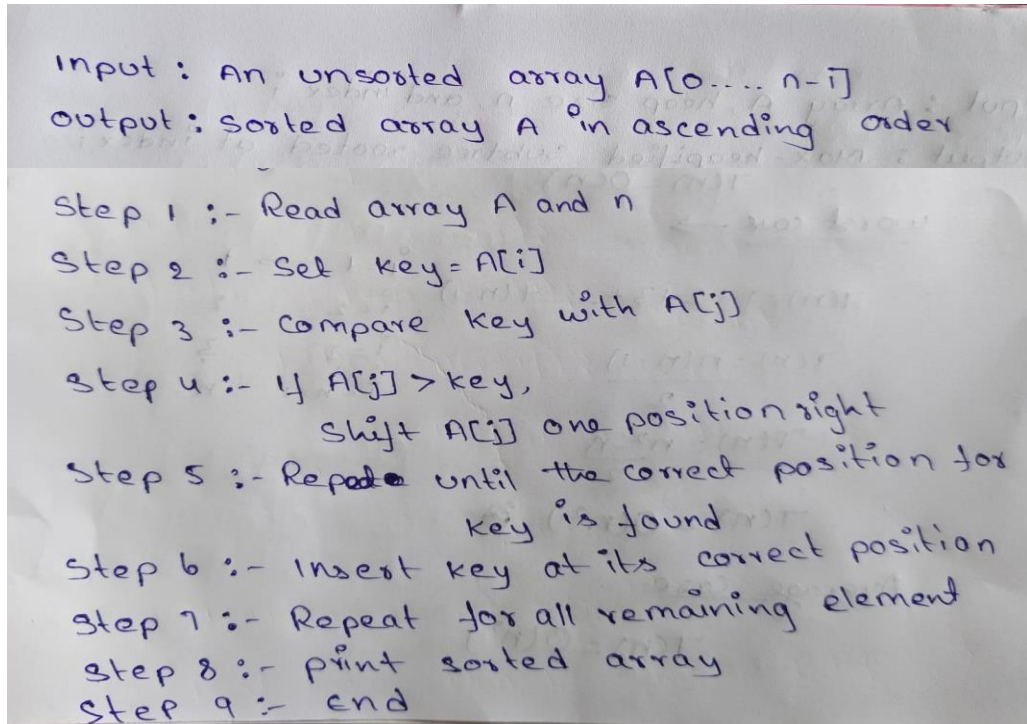
Time complexity:



for (i=0; i<n-1; i++) { $\rightarrow n-1$
 min=i; $\rightarrow (1)$
 for (j=i+1; j<n; j++) { $\rightarrow n$
 if (A[j] < A[min]) { $\rightarrow (1)$
 min=j; $\rightarrow (1)$
 }
 swap(A[i], A[min]); $\rightarrow (1)$
 }
 }
 $T(n) = (n-1)n$
 $= n^2 - n$
 $T(n) = O(n^2)$
Best case $\rightarrow O(n^2)$
Worst case $\rightarrow O(n^2)$
Average case $\rightarrow O(n^2)$

2) Insertion sort:

Algorithm:



Input : An unsorted array $A[0 \dots n-1]$
Output : Sorted array A in ascending order

Step 1 :- Read array A and n
Step 2 :- Set $key = A[i]$
Step 3 :- Compare key with $A[j]$
Step 4 :- If $A[j] > key$,
 Shift $A[j]$ one position right
Step 5 :- Repeat until the correct position for
 key is found
Step 6 :- Insert key at its correct position
Step 7 :- Repeat for all remaining element
Step 8 :- Print sorted array
Step 9 :- end

Program:

```
#include <iostream>
using namespace std;
class InsertionSort
{
public:
    int arr[100], n;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
    void logic()
    {
        int i, j, key;
        for (i = 1; i < n; i++){
            key = arr[i];
            j = i - 1;
            while (j >= 0 && arr[j] > key){
```



```
        arr[j + 1] = arr[j];
        j--;
    }

    arr[j + 1] = key;
}
}
void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
};

int main()
{
    InsertionSort is;

    is.arrayInput();

    cout << "Before Sorting: ";
    is.printArray();

    is.logic();

    cout << "After Sorting: ";
    is.printArray();

    return 0;
}
```

Output:



```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\insertionsort.exe"
Enter number of elements: 6
Enter the elements: 12
9
3
32
4
15
Before Sorting: 12 9 3 32 4 15
After Sorting: 3 4 9 12 15 32
```



Time Complexity:

```

for (i = 1; i < n; i++) { → n-1
    key = A[i]; → 1
    j = i - 1; → 1
    while (j >= 0 && A[j] > key) {
        A[j+1] = A[j]; → 1
        j--; → 1
    }
    A[j+1] = key; → 1
}

```

$T(n) = (n-1) + 1 + 1 + 1$
 $T(n) = n + 2$
 $T(n) = O(n) \rightarrow$ in Best case
 Ex: - $A = [1, 2, 3, 4, 5] \rightarrow$ while loop does not shift any element
 $T(n) = O(n)$
worst case →
 $T(n) = 1 + 2 + 3 + \dots + (n-1)$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = \frac{n^2 - n}{2}$
 $T(n) = O(n^2)$
Average case
 $T(n) = O(n^2)$

3) Merge Sort:

Algorithm:

Input: An unsorted array $A[0 \dots n-1]$
 Output: Sorted array A in ascending order

- Step 1: start
- Step 2: Read n and array A
- Step 3: Call mergesort($A, 0, n-1$)
- Step 4: If $low < high$, calculate $mid = (low + high) / 2$
- Step 5: Call mergesort(A, low, mid)
- Step 6: Call mergesort($A, mid + 1, high$)
- Step 7: merge the two sorted subarrays
- Step 8: Repeat until the complete array is sorted
- Step 9: Display sorted array
- Step 10: stop

Program:

```
#include <iostream>
using namespace std;
class MergeSort{
public:
    int arr[100],n;
    void arrayInput(){
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++){
            cin >> arr[i];
        }
    }
    void merge(int low, int mid, int high){
        int temp[100];
        int i = low;
        int j = mid + 1;
        int k = low;
        while (i <= mid && j <= high){
            if (arr[i] < arr[j]){
                temp[k] = arr[i];
                i++;
            }
            else{
                temp[k] = arr[j];
                j++;
            }
            k++;
        }
        while (i <= mid)
        {
            temp[k] = arr[i];
            i++;
            k++;
        }
        while (j <= high)
        {
            temp[k] = arr[j];
            j++;
            k++;
        }
        for (i = low; i <= high; i++)
        {
            arr[i] = temp[i];
        }
    }
    void logic(int low, int high)
    {
```

```
if (low < high)
{
    int mid = (low + high) / 2;
    logic(low, mid);
    logic(mid + 1, high);
    merge(low, mid, high);
}
}
void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
};

int main()
{
    MergeSort ms;

    ms.arrayInput();

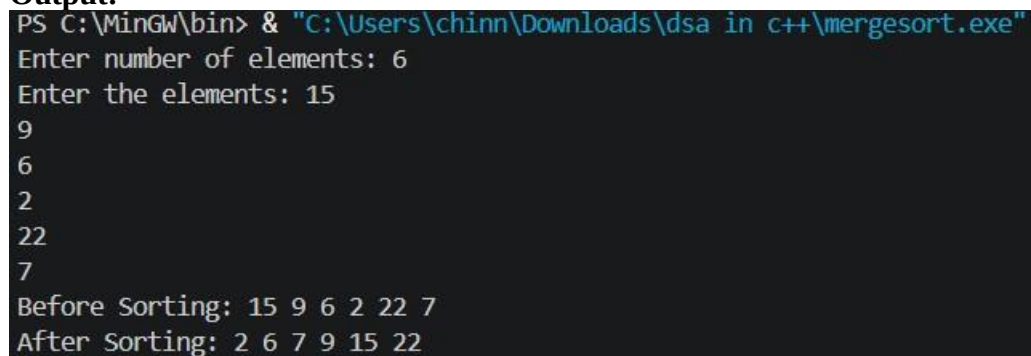
    cout << "Before Sorting: ";
    ms.printArray();

    ms.logic(0, ms.n - 1);

    cout << "After Sorting: ";
    ms.printArray();

    return 0;
}
```

Output:



```
PS C:\mingw\bin> & "C:\Users\chinn\Downloads\dsa in c++\mergesort.exe"
Enter number of elements: 6
Enter the elements: 15
9
6
2
22
7
Before Sorting: 15 9 6 2 22 7
After Sorting: 2 6 7 9 15 22
```



Time Complexity:

$\text{MergeSort}(A, \text{low}, \text{high}) \{ \rightarrow T(n)$
 $\quad \text{if } (\text{low} < \text{high}) \{ \rightarrow 1$
 $\quad \quad \text{mid} = (\text{low} + \text{high}) / 2 ; \rightarrow 1$
 $\quad \quad \text{mergeSort}(A, \text{low}, \text{mid}); \rightarrow T(\frac{n}{2})$
 $\quad \quad \text{mergeSort}(A, \text{mid} + 1, \text{high}); \rightarrow T(\frac{n}{2})$
 $\quad \quad \text{merge}(A, \text{low}, \text{mid}, \text{high}); \rightarrow n$
 $\quad \}$
 $\}$

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(\frac{n}{2}) + n & n > 1 \end{cases}$$

$$T(n) = 2T(\frac{n}{2}) + n$$

$T(n) = 2T(\frac{n}{2}) + n$
 $T(n) = 2[2T(\frac{n}{4}) + \frac{n}{2}] + n$
 $\quad = 2^2 T(\frac{n}{4}) + n + n$
 $\quad = 2^2 T(\frac{n}{4}) + 2n$
 $T(n) = 2^2 [2T(\frac{n}{8}) + \frac{n}{4}] + 2n$
 $\quad = 2^3 T(\frac{n}{8}) + n + 2n$
 $\quad = 2^3 T(\frac{n}{8}) + 3n$
 $T(n) = 2^3 [2T(\frac{n}{16}) + \frac{n}{8}] + 3n$
 $\quad = 2^4 T(\frac{n}{16}) + n + 3n$
 $\quad = 2^4 T(\frac{n}{16}) + 4n$
 $T(n) = 2^k T(\frac{n}{2^k}) + kn \Rightarrow$
 $\quad \quad \quad T(n) = n T(\frac{n}{n}) + n \log_2 n$
 $\quad \quad \quad = n(1) + n \log_2 n$
 $\quad \quad \quad = n + n \log_2 n$
 $\quad \quad \quad T(n) = \Theta(n \log_2 n)$
 $\quad \quad \quad \text{Best case} = O(n \log n)$
 $\quad \quad \quad \text{Worst case} = O(n \log n)$
 $\quad \quad \quad \text{Average case} = O(n \log n)$
 $n = 1$
 $\frac{n}{2^k} = 1$
 $n = 2^k$
 $\text{Apply log on both sides}$
 $\log n = \log 2^k$
 $\log n = k \log 2$
 $\frac{\log n}{\log 2} = k$
 $k = \log_2 n$

4) Quick Sort:

Algorithm:

Input : An unsorted array $A[0 \dots n-1]$
Output : Sorted array A in ascending order

Step 1 : Start
Step 2 : Read n and array A
Step 3 : call $\text{Quicksort}(A, 0, n-1)$
Step 4 : If $\text{low} < \text{high}$, select $\text{pivot} = A[\text{high}]$
Step 5 : set $i = \text{low} - 1$
Step 6 : set $j = \text{low}$
Step 7 : If $A[j] < \text{pivot}$, increment i and swap $A[i]$ and $A[j]$
Step 8 : Increment $j = j + 1$ and repeat step 7
Step 9 : swap $A[i+1]$ and $A[\text{high}]$
Step 10 : set $P = i + 1$
Step 11 : Apply Quick sort to left part
 $\text{Quicksort}(A, \text{low}, P-1)$
Step 12 : Apply Quick sort to right part
 $\text{Quicksort}(A, P+1, \text{high})$
Step 13 : Display sorted array
Step 14 : Stop.

Program:

```
#include <iostream>
using namespace std;
class QuickSort{
public:
    int arr[100], n;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++){
            cin >> arr[i];
        }
    }
    void printArray(){
        for (int i = 0; i < n; i++){
            cout << arr[i] << " ";
        }
    }
};
```

Name: Chinni Thabitha
Enrollment Number: 92460118038



```
    cout << endl;
}

int partition(int arr[], int lb, int ub)
{
    int pivot = arr[lb];
    int start = lb;
    int end = ub;

    while (start < end)
    {
        while (arr[start] <= pivot && start < ub)
        {
            start++;
        }

        while (arr[end] > pivot)
        {
            end--;
        }

        if (start < end)
        {
            swap(arr[start], arr[end]);
        }
    }

    swap(arr[lb], arr[end]);

    return end;
}

void quicksort(int arr[], int lb, int ub)
{
    if (lb < ub)
    {
        int location = partition(arr, lb, ub);

        quicksort(arr, lb, location - 1);
        quicksort(arr, location + 1, ub);
    }
}

void sort()
{
    quicksort(arr, 0, n - 1);
}

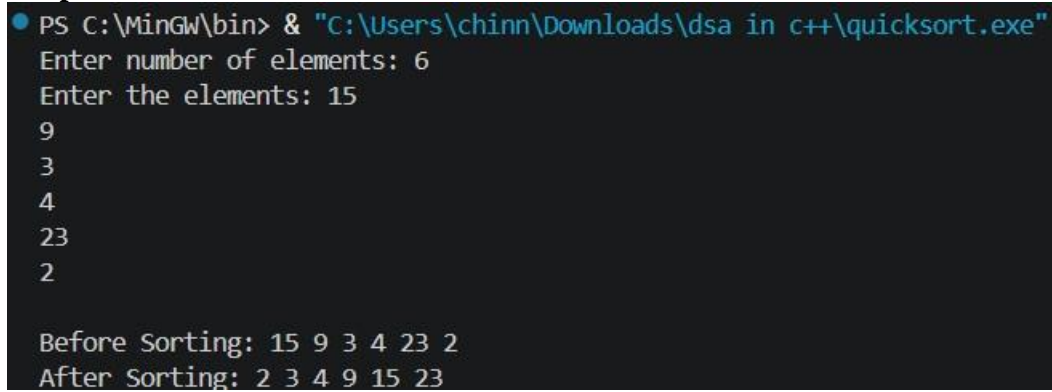
};

int main()
```



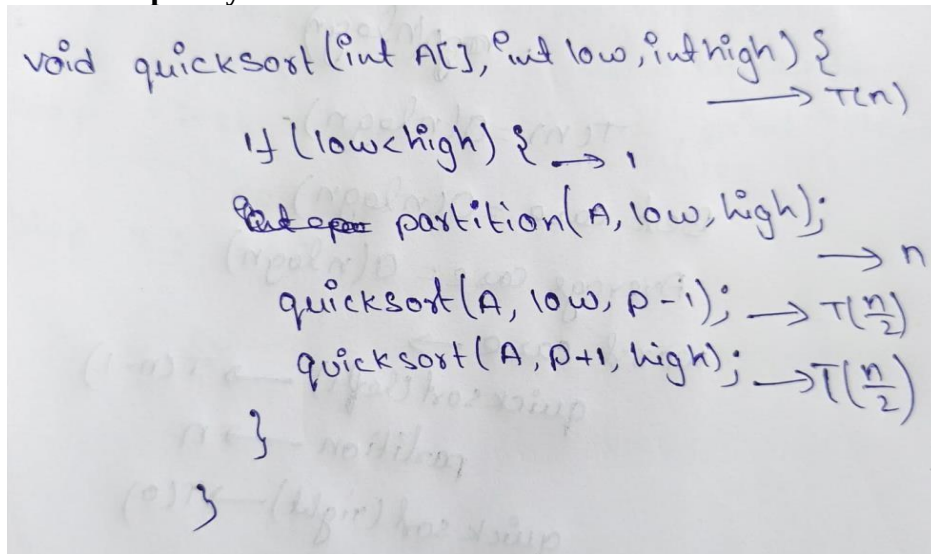
```
{  
    QuickSort qs;  
  
    qs.arrayInput();  
  
    cout << "\nBefore Sorting: ";  
    qs.printArray();  
  
    qs.sort();  
  
    cout << "After Sorting: ";  
    qs.printArray();  
  
    return 0;  
}
```

Output:



```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\quicksort.exe"  
Enter number of elements: 6  
Enter the elements: 15  
9  
3  
4  
23  
2  
  
Before Sorting: 15 9 3 4 23 2  
After Sorting: 2 3 4 9 15 23
```

Time Complexity:



```
void quicksort(int A[], int low, int high) {  
    if (low < high) {  
        partition(A, low, high);  
        quicksort(A, low, p-1);  
        quicksort(A, p+1, high);  
    }  
}
```

Annotations in the image:
- $T(n)$ above the initial call `quicksort(A, low, high)`
- 1 next to the `if (low < high)` condition
- n next to the `partition(A, low, high)` call
- $T(\frac{n}{2})$ next to the recursive call `quicksort(A, low, p-1)`
- $T(\frac{n}{2})$ next to the recursive call `quicksort(A, p+1, high)`

$$\begin{aligned}
 T(n) &= T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + n \\
 &= 2T\left(\frac{n}{2}\right) + n \\
 T(n) &= \begin{cases} 1 & n=1 \\ 2T\left(\frac{n}{2}\right) + n & n>1 \end{cases} \\
 T(n) &= 2T\left(\frac{n}{2}\right) + n \\
 a=2, \quad b=2 & \quad P>-1 \\
 k=1, \quad P=0 & \quad 0>-1 \\
 \log_a b &= \log_2 2 = 1 = k \\
 T(n) &= O(n^k \log^{P+1} n) \\
 &= O(n^1 \log^{0+1} n) \\
 &= O(n \log n) \\
 T(n) &= O(n \log n) \\
 \text{Best case} &= O(n \log n) \\
 \text{Average case} &= O(n \log n) \\
 \text{worst case} &\rightarrow \\
 \text{quicksort (left)} &\rightarrow T(n-1) \\
 \text{partition} &\rightarrow n \\
 \text{quicksort (right)} &\rightarrow T(0) \\
 T(n) &= T(n-1) + n \\
 T(n) &= n + (n-1) + (n-2) + \dots + 1 \\
 &= \frac{n(n+1)}{2} \\
 &= \frac{n^2 + n}{2} \\
 T(n) &= O(n^2)
 \end{aligned}$$

Conclusion:

The implementation and time analysis of Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort helps us understand how different sorting algorithms perform for different input sizes. By analyzing their best, average, and worst cases, we can choose the most suitable algorithm for a particular problem. Merge Sort and Quick Sort are generally faster for large datasets, while Bubble, Selection, and Insertion Sort are simpler and useful for small datasets.